ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

CENG 408

Discussion

AI / ML Modules: Disease Detection and LLM-Based Advisory

TARAS: Agricultural Analysis System

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

1. Introduction....3
1.1. Purpose of the Document....3
1.2. AI/ML Scope of the Project....3
2. Disease Detection Module....4
2.1. Problem Definition....4
2.2. Dataset and Data Preparation....5
2.3. Model Selection....7
2.4. Training Strategy....8
2.5. Experimental Results....9
2.6. Interpretation of Results....11
2.7. Comparison with Related Studies....12
2.8. Limitations of the Module....13
3. LLM Module....14
3.1. Problem Definition....14
3.2. System Design....14
3.3. Model Selection and Explanation....15
3.4. Benefits and Challenges....15
3.5. Comparison with Related Studies....16
3.6. Limitations of the Module....17
4. Future Work....17
4.1. Disease Detection Improvements....18
4.2. LLM Module Improvements....18
5. Conclusion....19
6.References....20

1. Introduction

1.1. Purpose of the Document

This chapter discusses the AI and machine-learning components of the TARAS precision-agriculture platform. Rather than restating implementation details already covered in the System Architecture and Methodology report, its purpose is interpretive: to justify the design decisions that shaped the two ML modules, to reflect on the results obtained from them, and to situate those results against comparable work in the literature. The discussion therefore focuses on why the chosen approaches are appropriate for the problem, what the experimental evidence tells us about their behaviour, and where the current design meets its limits.

Throughout the chapter the Disease Detection and LLM-based Advisory modules are treated as separate subsystems. Although both contribute to the same decision-support workflow, they address different kinds of problems, one visual and discriminative, the other linguistic and generative and they are therefore evaluated with different criteria and against different bodies of related work.

Where the discussion refers to specific results tables, performance metrics, or architectural diagrams, the reader is directed back to the corresponding sections of the main report. Where the current state of the system differs from the version originally documented in that report, those differences are made explicit.

1.2. AI/ML Scope of the Project

TARAS includes several data-driven subsystems, but only two are machine-learning components in the strict sense: the vision-based plant-disease detection module and the LLM-based advisory module. The disease detection module classifies leaf images into disease categories using a convolutional neural network, while the advisory module interprets sensor readings, irrigation outputs, disease results, and user input to generate natural-language guidance

The irrigation recommendation subsystem is excluded from this chapter. Although it is central to TARAS, it is based on calibrated agronomic calculations using reference evapotranspiration (ET_0), crop coefficient (Kc), and field sensor observations, following the FAO-56 methodology. Since this pipeline is deterministic and rule-based rather than learned from data, it is not evaluated here with classification metrics or learning-based arguments.

These two ML modules serve different purposes and are therefore discussed separately. Disease detection is a discriminative image-classification task, whereas the LLM advisory module is a generative, context-aware task that combines structured

system outputs with user text. Because they differ in input type, output form, and evaluation criteria, they are compared against different bodies of related work.

2. Disease Detection Module

2.1. Problem Definition

The problem addressed by the disease detection module is the automatic identification of plant diseases from a single photograph of a leaf taken under field conditions. Given an RGB image of a leaf, the model must assign one of a fixed set of disease or health labels and a confidence score associated with that label. The current version of the module covers twelve classes across five crop groups:

- Bell pepper
- Corn
- Potato
- Squash
- Tomato

The deployed label set are:

- Bell_pepper_bacterial_spot
- Bell_pepper_healthy
- Corn_common_rust
- Corn_northern_leaf_blight
- Potato_Leaf_blight
- Squash_powdery_mildew
- Tomato_healthy
- Tomato_Late_blight
- Tomato_Leaf_blight
- Tomato_Leaf_Mold
- Tomato_Septoria_leaf_spot
- Tomato_Spider_mites_Two_spotted_spider_mite.

This task is important because early disease identification is one of the most cost-effective interventions in crop management. Diseases such as late blight, bacterial spot, early blight, and leaf mold can spread quickly and cause serious yield loss if they are not addressed in time. Since many small-scale farmers do not have routine access to an agronomist, a tool that turns a smartphone photograph into a structured diagnosis can reduce the delay between noticing symptoms and taking action.

The main difficulty is that field images are highly variable. Photos may be taken under different lighting conditions, from different angles, with partial occlusion and complex natural backgrounds. In addition, several tomato diseases produce visually

similar symptoms, especially in early stages. Dark leaf spots, for example, may correspond to bacterial spot, early blight, Septoria leaf spot, or target spot. Distinguishing them requires attention to subtle visual cues such as lesion shape, margins, concentric patterns, and small structures like pycnidia. For this reason, a model trained only on clean laboratory images may not perform reliably in real field conditions.

Within TARAS, the disease detection module has two roles. First, it works as a standalone diagnostic tool: the user uploads a leaf image, and the system returns a class label and confidence score. Second, its output is used by the LLM-based advisory module, which can combine the detected disease with recent humidity, irrigation, and field history to produce contextual guidance. Therefore, the module must be accurate enough to be useful on its own and structured enough to support downstream reasoning.

2.2. Dataset and Data Preparation

The disease detection module is trained on a hybrid dataset that combines laboratory and real-world image sources. The laboratory component comes from PlantVillage, a public dataset of healthy and diseased leaf images captured under controlled conditions, usually with a single leaf placed against a plain background. Although this dataset is large and clean, it does not fully represent field conditions.

To reduce this limitation, TARAS also uses real-world image sources. The first is a curated RW dataset assembled for this project from permissively licensed sources such as iNaturalist, Wikimedia Commons, educational extension services, and other open repositories. Each image is tracked with metadata including its source URL, licence, label confidence, and visual difficulty. The second is PlantDoc, a real-world plant disease dataset containing images with natural backgrounds, varied lighting, multiple leaves, and other field-like conditions. Together, these sources make the training data more representative of real deployment scenarios.

The reason for combining these datasets is the domain-shift problem in plant-disease classification. Models trained only on PlantVillage often perform very well on held-out PlantVillage images but perform much worse on field photographs. In this sense, PlantVillage is large but unrepresentative, while real-world datasets are more realistic but smaller and noisier. The hybrid dataset is designed to benefit from both: the scale of PlantVillage and the field variability of RW and PlantDoc images.

The real-world collection process prioritises both class balance and source diversity. The target is at least 100 real-world images per class, with 300 as a preferred goal when available. Images are reviewed for symptom clarity, background complexity, visibility of the leaf, and label reliability. Scarce classes are prioritised during collection,

and duplicates or near-duplicates are removed using perceptual hashing to prevent the model from learning repeated images or source-specific patterns.

The same preprocessing pipeline is applied across all data sources. Images are checked for format and resolution, resized consistently, standardised in colour space, and normalised using the parameters expected by the backbone network. The same normalisation is also used during inference. Data augmentation, such as random flips, small rotations, brightness and contrast changes, and mild crop or zoom, is applied only to the training split. Validation and test images are not augmented, and they are drawn from real-world data so that reported performance reflects field-like conditions rather than clean laboratory images.

The final training dataset used for the current production candidate contains 5,913 training images across 12 classes. The training split is distributed as follows:

Class Name	Sample
Bell_pepper_bacterial_spot	210
Bell_pepper_healthy	200
Corn_common_rust	232
Corn_northern_leaf_blight	381
Potato_Leaf_blight	447
Squash_powdery_mildew	222
Tomato_healthy	714
Tomato_Late_blight	686
Tomato_Leaf_blight	706
Tomato_Leaf_Mold	694
Tomato_Septoria_leaf_spot	715

Tomato_Spider_mites_Two_spotted_spider_mite	706

The validation design intentionally uses two separate splits. The first, val_pd, contains 182 images and is treated as the more realistic validation source. The second, val_pv, contains 518 images and reflects the easier PlantVillage-like domain. During model selection, TARAS uses a weighted combined score of 0.65 x val_pd macro F1 plus 0.35 x val_pv macro F1 so that real-world behaviour dominates checkpoint selection without discarding the stabilising value of the cleaner validation domain.

Two test sets are maintained. The first, test_pd, contains 244 images and is the principal real-world evaluation split used for deployment decisions. The second, test_balanced, contains 275 images and was created by adding extra PV examples to class categories in order to reduce the variance of test results across all classes.

2.3. Model Selection

The final deployment candidate selected for TARAS is Swin Small Transformer (swin_small_patch4_window7_224). This differs from earlier stages of the project, where EfficientNetV2 variants were used as the main baseline and an EfficientNetV2-M curriculum-trained checkpoint was considered the strongest option. After the label schema was revised and the merged 12-class dataset was finalized, the models were re-evaluated under the updated setting.

In the revised comparison experiment, ConvNeXt-Tiny, ConvNeXt-Small, EfficientNetV2-rm-M, and Swin-Tiny were trained using the same recipe. ConvNeXt-Small achieved the best result in that comparison run, with a test_pd macro F1 score of 0.8953. However, a later focused experiment with Swin-Small, using the same revised dataset and evaluation logic, produced stronger real-world performance and became the final deployment candidate.

The selection of Swin-Small is based on its overall performance and stability. In the final production-candidate run, it achieved 0.9344 accuracy and 0.9269 macro F1 on the test_pd set, outperforming the strongest models from the earlier comparison. It also performed well on the test_balanced set, reaching 0.9418 accuracy and 0.9412 macro F1. These results suggest that Swin-Small generalizes well across both real-world and balanced evaluation conditions.

Swin-Small is also suitable for the task from a modelling perspective. After label merging, the main challenge is less about broad class separation and more about fine-grained visual differences between similar diseases. Windowed self-attention allows the model to capture local lesion details while still considering broader leaf structure,

which is useful for difficult pairs such as Potato_Leaf_blight vs. Tomato_Late_blight and Tomato_Leaf_blight vs. Tomato_Septoria_leaf_spot.

For deployment, the trained PyTorch model is converted to ONNX and served on AWS Lambda using ONNX Runtime. This replaced the earlier TensorFlow/Keras and TFLite path because it supports smaller container images, simpler CPU-only inference, and a clearer separation between training and serving. Before promotion, PyTorch and ONNX outputs are checked on fixture images, with a maximum absolute probability difference below 10^{-3} required for deployment.

2.4. Training Strategy

The current system uses a unified, single-stage training strategy instead of the earlier curriculum-based approach. This choice makes the pipeline simpler, more stable, and easier to reproduce while still allowing the model to learn from both laboratory and real-world images.

The final training set combines PlantVillage (PV) images with the curated real-world (RW) dataset. This exposes the model to both clean, label-consistent laboratory images and more variable field-style images during the same training process. Instead of applying a staged transition from PV to RW data, domain balance is handled through sampling and loss weighting.

To reduce class imbalance, the training pipeline uses a WeightedRandomSampler, which increases the chance of selecting underrepresented classes during training. In addition, class-weighted cross-entropy loss is applied, with weights computed inversely to class frequency. This prevents the model from being dominated by high-frequency classes and helps each disease category contribute more fairly to the learning process.

Data augmentation is applied only to the training split. The pipeline includes random resized cropping, horizontal flipping, rotation, and mild brightness and contrast changes. These augmentations improve robustness to differences in viewpoint, lighting, and background while preserving the disease-specific visual patterns needed for classification.

The evaluation protocol is explicitly designed to reflect deployment conditions. Two separate test sets are maintained:

- **test_pd:** composed entirely of real-world images, representing the primary benchmark for deployment performance.
- **test_balanced:** a more class-balanced evaluation set, constructed to analyse per-class behaviour under controlled distribution conditions.

Model selection is based on a weighted validation score computed from two validation sets:

- **val_pd (real-world dominant)**, assigned a higher weight (% 0.65)
- **val_pv (clean distribution)**, assigned a lower weight (% 0.35)

This weighting ensures that the selected model prioritises real-world generalisation while still retaining sensitivity to clean, canonical visual patterns.

Instead of using a multi-stage curriculum, the current approach trains the model on all data sources at the same time. The balance between clean laboratory images and real-world images is handled through sampling and loss weighting, rather than switching between different training phases. At the same time, it reduces the overall complexity of the training pipeline, making the system easier to reproduce and maintain.

2.5. Experimental Results

The most important numerical result of this work is the distinction between the final deployment candidate and the broader comparison set. The selected deployment model, Swin-Small, achieved 0.9344 accuracy and 0.9269 macro F1 on the primary real-world test set test_pd, and 0.9418 accuracy and 0.9412 macro F1 on the secondary test_balanced split. In the same run, the weighted F1 on test_pd was 0.9338, with macro precision 0.9330 and macro recall 0.9243.

Split	Accuracy	Macro F1	Loss
test_pd	0.9344	0.9269	0.5502
test_balanced	0.9418	0.9412	0.5502

The benchmark comparison on the revised dataset produced the following ranking on test_pd macro F1: ConvNeXt-Small 0.8953, ConvNeXt-Tiny 0.8934, EfficientNetV2-rm-M 0.8783 and Swin-Tiny 0.8533. These results are still strong, but they establish that the separately tuned Swin-Small deployment run outperformed every model from the comparison set. This is why the document distinguishes between the benchmark comparison run and the selected deployment checkpoint.

Model	Test_PD Acc	Test_PD Macro F1	Test_Balanced Macro F1
ConvNeXt-Small	0.9016	0.8953	0.9151
ConvNeXt-Tiny	0.9057	0.8934	0.9158
EfficientNetV2-rm-M	0.8811	0.8783	0.9011
Swin-Tiny	0.8730	0.8533	0.8848
Swin-Small	0.9344	0.9269	0.9412

Per-class results of the final Swin-Smal model show that:

Test_PD_Results:

Classification Report:

Precision	Recall	f1-score	Support
Bell_pepper_bacterial_spot	0.8000	0.8889	0.8421
Bell_pepper_healthy	0.9000	0.8182	0.8571
Corn_common_rust	1.0000	0.9000	0.9474
Corn_northern_leaf_blight	0.9167	1.0000	0.9565
Potato_Leaf_blight	0.9167	0.7333	0.8148
Squash_powdery_mildew	1.0000	1.0000	1.0000
Tomato_Late_blight	0.9310	1.0000	0.9643
Tomato_Leaf_Mold	1.0000	0.9688	0.9841
Tomato_Leaf_blight	0.8649	0.9143	0.8889
Tomato_Septoria_leaf_spot	0.9333	0.9032	0.9180
Tomato_Spider_mites_Two_spotted_spider_mite	1.0000	1.0000	1.0000
Tomato_healthy	0.9333	0.9655	0.9492

Test_Balanced_Results:

Classification Report:

Precision	Recall	f1-score	Support
Bell_pepper_bacterial_spot	0.8667	0.9286	0.8966
Bell_pepper_healthy	0.9286	0.8667	0.8966
Corn_common_rust	1.0000	0.9412	0.9412
Corn_northern_leaf_blight	0.9412	1.0000	0.9697
Potato_Leaf_blight	0.9375	0.7895	0.8571
Squash_powdery_mildew	1.0000	1.0000	1.0000
Tomato_Late_blight	0.9310	1.0000	0.9643
Tomato_Leaf_Mold	1.0000	0.9688	0.9841
Tomato_Leaf_blight	0.8649	0.9143	0.8889
Tomato_Septoria_leaf_spot	0.9333	0.9032	0.9180
Tomato_Spider_mites_Two_spotted_spider_mite	1.0000	1.0000	1.0000
Tomato_healthy	0.9333	0.9655	0.9492

2.6. Interpretation of Results

The revised results support a more balanced conclusion than earlier project stages. The main finding is not that one architecture is always superior, but that label design, dataset quality, and model selection strongly affect final performance.

The comparison also shows that architecture alone does not determine success. Candidate models were trained and evaluated under the same experimental setup, and the final classifier was selected based on measured performance rather than assumptions about a specific architecture family. This makes the selection process more consistent with the deployment goal of the project.

Evaluation was also treated carefully. The real-world test split, test_pd, was used as the main indicator of deployment readiness, while test_balanced was used as a secondary sanity check. Swin-Small achieved 0.9269 macro F1 on test_pd and 0.9412 macro F1 on test_balanced. The small gap between these results suggests that the model performs well not only on the balanced split, but also on the harder field-oriented test set.

The remaining errors are understandable. For example, confusion between Potato_Leaf_blight and Tomato_Late_blight suggests that the model sometimes focuses

more on disease-like visual patterns than crop identity. This is a common limitation in plant-disease classification and does not mean the model is unusable. Instead, it shows the boundary of what can be inferred from a single leaf image without additional crop metadata or multiple images.

Overall, the revised experiments show that the final TARAS classifier is not only a laboratory benchmark model. It is a deployment-oriented classifier selected after dataset revision, label-schema redesign, model comparison, and error analysis.

2.7. Comparison with Related Studies

The classic reference point remains Mohanty, Hughes and Salathe (2016), who trained deep CNNs on 54,306 PlantVillage images and reported 99.35% accuracy on a held-out test set.[1] That result established the feasibility of deep learning for plant disease detection, but it also exemplifies the main limitation that later studies exposed: excellent laboratory accuracy does not guarantee strong field performance.[2] TARAS deliberately accepts lower headline numbers in exchange for evaluation on more realistic imagery and therefore addresses a different question than PlantVillage-only studies.

A useful field-oriented comparison is PlantDoc and later work built around it. Introduced PlantDoc as a challenging real-world benchmark with cluttered backgrounds and cross-domain variability [3], and later object-detection and classification studies continued to use it precisely because it is harder than PlantVillage. Relative to this line of work, TARAS is similar in spirit because it prioritises natural images, but it differs in being a task-specific hybrid corpus with label redesign guided by error analysis rather than a fixed public benchmark alone.

For comparison with modern transformer-based approaches,recent studies based on Swin Transformer architecture have demonstrated strong performance under natural and noisy image conditions.[4] TARAS is aligned with these findings: the final selection of Swin-Small over the revised 12-class dataset supports the claim that transformer-style feature modelling can be advantageous once the label schema and data preparation are appropriate.

The main point of difference is methodological. Most published papers compare architectures on a fixed benchmark, whereas TARAS also redesigns the label taxonomy when the benchmarked error patterns show that a finer distinction is not reliable enough for deployment. This is why the final document reports not only architecture comparison but also the decision to merge the two potato blight classes. For a live agricultural system, this decision is more meaningful than preserving a more granular taxonomy that produces unstable advice.

Taken together, the literature suggests that TARAS occupies an intermediate position. It does not chase PlantVillage-only accuracy numbers, and it does not depend

exclusively on one public field dataset either. Instead, it uses a curated hybrid dataset, a deployment-oriented test split and a model-selection process that is directly tied to the target application

2.8. Limitations of the Module

Several limitations follow from the current design and should be acknowledged clearly. First, the classifier is limited to twelve categories across five crops. Images outside this scope, such as different crops, unseen diseases, nutrient deficiencies, pest damage, or abiotic stress, may still be forced into one of the known classes. Since the model does not currently include an out-of-distribution rejection class, the backend confidence threshold is the only mechanism for reducing unreliable predictions, and this is not a perfect solution.

Second, the real-world dataset is still relatively small. Although source diversity was considered during collection, the dataset reflects the regions, seasons, and crop conditions available in permissively licensed image sources. As a result, the model is likely to perform better in contexts similar to the training data and less reliably in underrepresented conditions, such as specific greenhouse cases in Türkiye. Expanding the real-world dataset remains an important future improvement.

Third, some classes remain weaker because of dataset imbalance. PlantVillage contains fewer examples for classes such as Tomato Mosaic Virus and Potato Healthy, and augmentation cannot fully replace real image diversity. The curated real-world data helps reduce this issue, but these classes still represent weak points in the current model.

Fourth, the module works at the image level. It classifies one leaf image at a time and does not reason across multiple images of the same plant or track symptom progression over time. This limits performance in early-stage disease cases, where a single leaf may be ambiguous but several images over time could make the diagnosis clearer.

Finally, the module is only a visual diagnostic aid. Some plant-health problems, such as over-irrigation, nutrient deficiency, or root disease, may produce visible symptoms without being identifiable from the image alone. The LLM-based advisory module partially addresses this by combining disease predictions with sensor and irrigation history, but the image classifier should not be treated as a standalone diagnostic authority. Its role is to narrow down possible causes and support further investigation.

3. LLM Module

3.1. Problem Definition

The LLM-based advisory module addresses a different but complementary problem to disease detection: it turns system outputs into explanations that users can understand and act on. Soil moisture values, irrigation recommendations, and disease confidence scores are useful only if the user can interpret what they mean for their field. Since TARAS is designed for non-expert users, this translation layer is an important part of the platform.

The module answers questions such as “What do these values mean for my field today?” and “What should I do next?” using the context of the user’s farm, field, zone, and crop. Its responses are generated from structured inputs such as live sensor readings, recent irrigation decisions, disease detection results, and user-provided information. The goal is not to create an open-ended chatbot, but a decision-support assistant whose answers remain grounded in the data available in the system.

The main design challenge is balancing usefulness and reliability. The module needs to be flexible enough to answer different user questions, but constrained enough to avoid common LLM problems such as hallucinated details, overconfident statements, or plausible but incorrect recommendations. Therefore, the advisory system is designed to produce practical guidance while keeping its reasoning tied to concrete data sources and curated agricultural knowledge.

3.2. System Design

The advisory module is implemented as an agentic pipeline instead of a single-shot prompt. When the user asks a question, the model does not answer only from its general training knowledge. It first decides whether it needs to call one or more read-only tools to ground the answer. These tools can retrieve live database records, search the curated agricultural knowledge base, or direct the mobile app to a relevant screen when visual information is more useful than text. The final response is generated only after the tool results are returned.

The agent currently has access to nine tools. These include tools for mobile navigation, field and zone summaries, latest sensor readings, historical zone data, irrigation history, sensor diagnostics, carbon summaries, and knowledge-base search. Each tool has a strict input schema, enforces user-specific data scope, and is audit-logged. To keep the system controlled, the agent is limited to three tool-use iterations, eight tool calls per request, a thirty-second timeout, and capped tool results.

Domain grounding is mainly provided through the search_knowledge tool. This tool uses PostgreSQL full-text search over a curated knowledge base of about 146 fragments covering crop guides, diseases, pests, soils, fertilizers, irrigation practices, climate considerations, and system usage. Instead of relying only on the LLM’s internal

knowledge for agronomic advice, the system retrieves relevant project-controlled guidance and lets the model reason over it. This also allows the advisory knowledge to be updated without retraining the model.

The second grounding mechanism is the system prompt. It is compact, cached, and designed to enforce a few key behaviours: answer in the user's language, stay short and direct unless more detail is needed, use tools for user-specific data, and use navigation only when a visual screen would genuinely help. This text-first design was chosen because earlier versions overused navigation and interrupted the user's flow.

3.3. Model Selection and Explanation

The production model is Anthropic Claude Haiku 4.5 (claude-haiku-4-5-20251001), accessed through the Anthropic TypeScript SDK. Haiku 4.5 was chosen because the advisory module mainly requires short, context-grounded responses rather than long open-ended generation. It is faster and more cost-effective than larger models, especially with system-prompt caching enabled.

The reliability of the module comes less from the model itself and more from the surrounding architecture: the tool-use loop, scoped read-only database access, curated retrieval corpus, rate limiting, audit logging, and compact system prompt. This makes the design mostly model-agnostic. The system also includes a fallback mode using Qwen3 through Groq when the Anthropic provider is unavailable, although this fallback uses a simpler prompt and does not include the full tool loop.

The agentic design was selected over simpler alternatives such as a single-shot prompt or fixed retrieval pipeline. Different user questions require different parts of the system state: soil-moisture questions may need zone history, disease-related questions may need irrigation history, and explanation questions may need the knowledge base. Instead of inlining all possible context into every request, the tool-loop lets the model retrieve only what is relevant. This keeps responses cheaper, more focused, and easier to trace through the selected tool calls.

3.4. Benefits and Challenges

The main benefit of the advisory module is accessibility. Raw values such as soil moisture, crop coefficient, or disease confidence scores are difficult for non-expert users to interpret on their own. The advisory module turns these outputs into practical explanations, such as why a specific zone needs irrigation today while another does not. In this sense, it makes the platform's technical outputs usable for the intended audience.

The module also acts as an integration layer across TARAS subsystems. Disease results, irrigation recommendations, sensor readings, and field history are stored separately, but user questions often combine them. For example, a question such

as “Is this leaf spot related to overwatering?” requires disease output, irrigation history, and humidity data together. The tool-loop design allows the module to retrieve and connect these pieces without requiring the user to understand the underlying database structure.

Another benefit is explainability. Because responses are grounded in tool outputs and retrieved knowledge, the module can refer to the specific factors behind an answer, such as current soil moisture, growth stage, irrigation history, or a relevant disease guide. This makes the response more trustworthy than a generic chatbot answer.

The main challenges are accuracy, latency, cost, reliability, and trust boundaries. The grounding mechanisms reduce the risk of hallucination, but they do not eliminate it, especially for edge cases outside the curated knowledge base. The agentic tool loop also adds latency because some questions require multiple tool calls, although streaming improves perceived response time. Cost is controlled through rate limits and result-size caps, while provider outages are handled through the Anthropic/Groq fallback setup. Finally, the advisory output must be framed as decision support, not as a replacement for expert diagnosis, especially for high-risk cases such as possible late blight.

3.5. Comparison with Related Studies

Recent studies on LLMs in agriculture, such as Wang et al. (2024), identify the same problem addressed by the TARAS advisory module: smart-farming systems can produce useful analytical outputs, but farmers may struggle to interpret them. LLMs can help by translating sensor readings, model outputs, and agronomic recommendations into natural-language explanations. TARAS follows the retrieval-augmented generation (RAG) approach rather than fine-tuning: Claude Haiku 4.5 is grounded with a curated agricultural knowledge base retrieved through PostgreSQL full-text search.

Most agricultural chatbot systems fall into two broad groups. Rule-based systems are predictable and explainable, but they are limited to predefined questions. Free-form LLM chatbots are more flexible, but they can produce unsupported or incorrect advice. TARAS is designed to sit between these two approaches. Users can ask questions naturally, but the model is constrained by tool use, scoped live data access, and retrieval grounding.

The TARAS advisory module differs from many agricultural LLM systems in three main ways. First, it connects to live sensor, irrigation, disease, diagnostic, and carbon data through scoped and audit-logged tools. Second, it remains text-first and uses navigation only when a screen view would help the user more than a written answer. Third, it includes a provider fallback path, so the advisory function can continue in a simpler form if the primary LLM provider is unavailable.

3.6. Limitations of the Module

The advisory module depends heavily on the quality of its upstream data. If a zone has faulty sensors, stale readings, or inconsistent metadata, the generated response may also be unreliable, even if it sounds fluent. The get_sensor_diagnostics tool helps reduce this risk by allowing the agent to check sensor-health information and add caveats when needed, but the module cannot be more accurate than the data it receives.

Retrieval quality is another limitation. The curated knowledge base contains about 146 fragments across eight categories, so it is useful but not encyclopedic. Questions that match its coverage can be grounded well, while questions outside its scope may rely more on the model's general knowledge. For this reason, expanding and maintaining the knowledge base remains an important future task.

The module should also not be treated as a replacement for expert agronomic judgment. In complex or high-risk cases, such as unusual disease symptoms, multiple stress factors, or decisions with serious economic consequences, the advisory response should be used as a starting point rather than a final authority.

The agentic tool loop introduces some tool-specific risks, such as selecting the wrong tool, misreading a tool result, or answering with incomplete context. These risks are reduced through iteration limits, result-size caps, scope checks, and audit logging, but they cannot be removed completely.

Finally, the module requires an internet connection. This may be a limitation in rural areas with unstable connectivity. At present, the mobile client can show the last-known advisory state, but it cannot generate new advisory responses offline. An offline-capable lightweight model could be considered as future work.

4. Future Work

Both ML modules are at a point where the core design is stable and the remaining work is largely incremental, extending coverage, improving robustness, and closing specific gaps identified during development. The items below are grouped by module and ordered loosely by estimated impact on the user experience.

4.1. Disease Detection Improvements

Future improvements for the disease-detection module should build on the revised 12-class system rather than returning to a broader but less reliable label set. The first priority is targeted dataset expansion for the hardest categories, especially Potato_Leaf_blight and visually overlapping classes. Adding more real-world images

from Turkish greenhouse and open-field conditions would likely improve performance more than switching to a new backbone.

A second priority is confidence calibration and rejection handling. Although the current model gives strong top-1 predictions, production use would benefit from calibrated thresholds and a fallback path for uncertain cases. For example, the system could return “low confidence, please capture another image or use the advisory module” instead of always forcing a class.

Future architecture experiments should also be selective. The current results already identify Swin-Small as a strong deployment candidate, with credible alternatives such as ConvNeXt-Small. Therefore, the next step should focus on improving the selected Swin-Small pipeline and occasionally testing it against a small number of strong baselines.

4.2. LLM Module Improvements

The LLM advisory module has four main future improvements.

First, the knowledge base should be expanded. The current 146 curated fragments cover common cases well, but unusual pest–disease interactions, region-specific crop guidance, and less common soil types need more coverage. New content should remain curated and attributable rather than scraped from the open web.

Second, the tool set can be extended. Useful additions include a weather-forecast tool, a historical-analogue tool for comparing current conditions with similar past cases, and a push-notification tool for proactive alerts.

Third, offline and low-bandwidth support should be considered. A small on-device fallback model could answer a narrow set of common questions using cached field data, such as irrigation status, recent disease results, and basic crop-stage guidance.

Fourth, the module needs a feedback loop. A simple thumbs-up/thumbs-down mechanism would help identify weak answers, improve the system prompt, and guide future knowledge-base updates.

Additional improvements could include conversation summarisation, voice input, what-if irrigation scenarios, and more detailed per-tool authorization.

5. Conclusion

This chapter discussed the two AI/ML components of TARAS: the vision-based disease detection module and the LLM-based advisory module. Although they solve different problems, both support the same goal: helping users interpret field data and make better agricultural decisions.

For disease detection, the project converged on a revised 12-class label schema and selected Swin-Small as the final deployment candidate after dataset revision, error analysis, and comparative retraining. In the deployment-oriented run, Swin-Small achieved 0.9344 accuracy and 0.9269 macro F1 on the primary real-world test split, and 0.9418 accuracy and 0.9412 macro F1 on the secondary balanced split. These results show that the strongest improvements came not only from model architecture, but from aligning the label schema, dataset, and evaluation protocol with real deployment conditions.

The LLM advisory module addresses a different need: turning structured system outputs into understandable guidance. Its agentic tool-loop design, scoped read-only tools, curated knowledge base, and provider fallback were chosen to balance usefulness with reliability. The module has been demonstrated end-to-end in production, with typical response times of three to six seconds, but it has not yet been evaluated through a large-scale quantitative study. Its current evaluation is therefore mainly qualitative, based on coherence, grounding, and consistency with system outputs.

The current limitations are clear: the classifier is limited to twelve classes, the advisory knowledge base is compact, the LLM module depends on cloud connectivity, and the classifier does not yet provide visual explanation maps. However, these are incremental improvement areas rather than reasons to redesign the system from scratch.

Overall, the AI/ML work in TARAS follows a consistent design philosophy: both modules should be useful without overclaiming, flexible without becoming ungrounded, and supportive without replacing expert judgment. The disease classifier and advisory agent are therefore best understood as decision-support tools that help non-expert users make better use of the data produced by their own fields.

6.References

[1] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016).
Using Deep Learning for Image-Based Plant Disease Detection.
https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2016.01419/full

[2] Barbedo, J. G. A. (2018).
Impact of dataset size and quality on deep learning-based plant disease classification.
https://www.researchgate.net/publication/326957145_Impact_of_dataset_size_and_variety_on_the_effectiveness_of_deep_learning_and_transfer_learning_for_plant_disease_classification

[3] Singh, D., Jain, N., Jain, P., et al. (2020).
PlantDoc: A Dataset for Visual Plant Disease Detection.
https://dl.acm.org/doi/epdf/10.1145/3371158.3371196

[4]Liu, Z., Lin, Y., Cao, Y., et al. (2021).
Swin Transformer: Hierarchical Vision Transformer using Shifted Windows.
https://arxiv.org/pdf/2103.14030